

Fragmentación y recuperación

MMIA 6013 — IA Generativa y Agentes · Sesión 08 · miércoles 23 de septiembre de 2026

Semana 2 · RA-2 · de los 110 minutos, 70 son actividad y 40 son este deck

Daniel Andrés Riofrío Almeida

Ayer prometí el cuello de botella. Aquí está

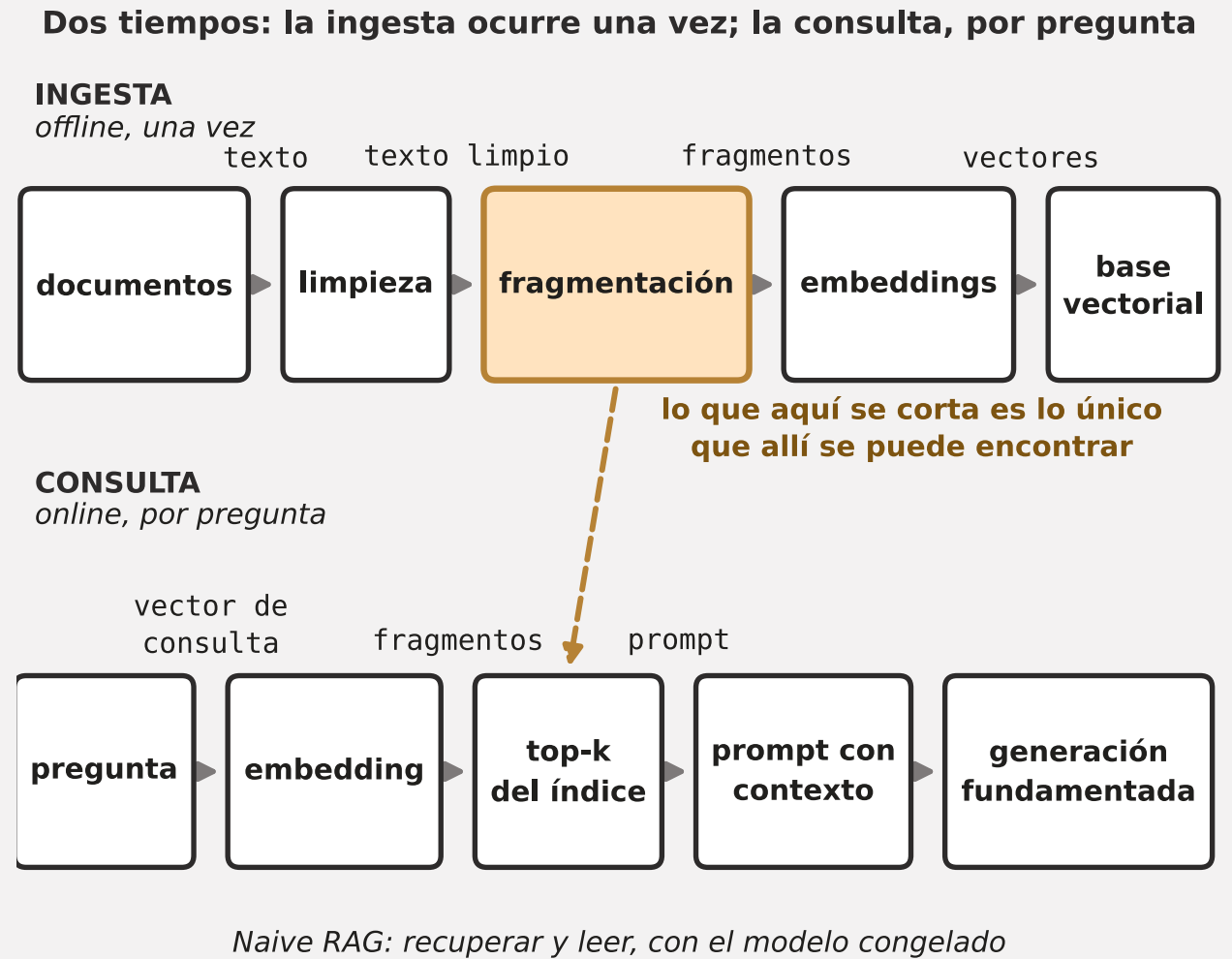
- Cerré ayer así: «mañana: fragmentar y recuperar — y ahí está el cuello de botella de verdad»
- Hoy el índice es una caja negra: devuelve los k fragmentos más parecidos y ya
- Lo nuevo es una idea con dos caras: el fragmento es la unidad de todo el sistema...
- ...y ninguna forma de buscar arregla un fragmento mal cortado
- Dos mitades de la recuperación aumentada con generación (RAG): cómo se corta y cómo se busca
- Hoy NO se mide nada: Recall@k, MRR, Hit Rate y golden sets son mañana

RAG: lo que la sigla promete y lo que no

08

RAG no le enseña nada al modelo: le cambia lo que tiene delante

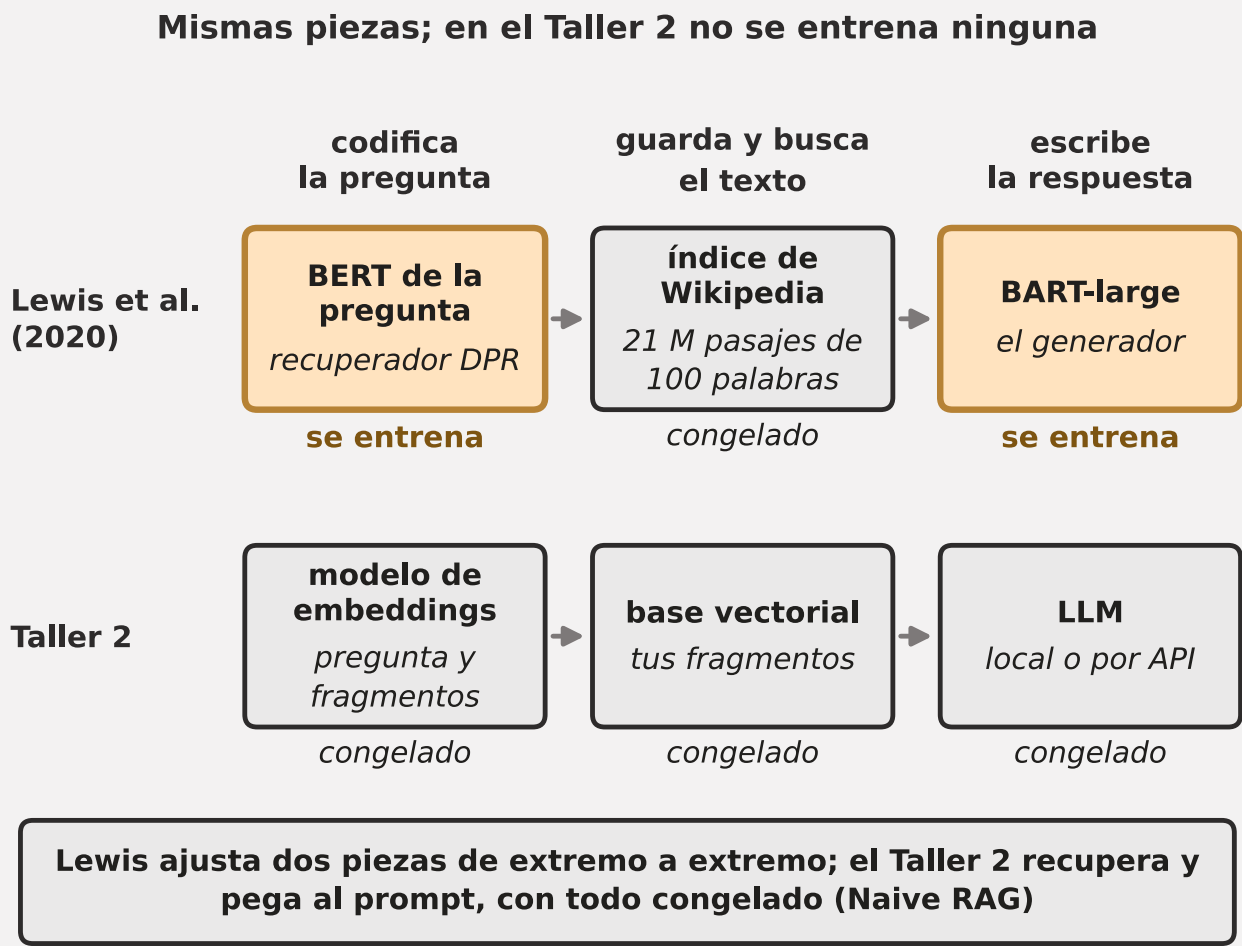
- Tres motivos: conocimiento congelado, dominio privado y alucinación
- Ingesta, offline y una vez: documentos → limpieza → FRAGMENTACIÓN → embeddings → base vectorial
- Consulta, online y por pregunta: pregunta → embedding → top-k → prompt con contexto → generación fundamentada
- Gao lo llama Naive RAG: el marco «Retrieve-Read», con el modelo congelado
- Cuando NO: si el documento cabe cómodamente en el prompt, pásalo directo
- Si lo que quieres cambiar es el estilo y no el conocimiento, eso es ajuste fino (semana 4)



Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §1, líneas 7–20; [curso/actividades/semana-2/s2-mie-explicacion.html](#), líneas 39–44, 46–53 y 55–60; Gao et al. (2023), §II.A, p. 3 (Naive RAG, «Retrieve-Read») y §II.D, p. 5 (RAG frente a ajuste fino); Lewis et al. (2020), p. 1; Alammar & Grootendorst (2024), cap. 8, §«From Search to RAG», p. 250 (generación fundamentada); [proceso/contexto/glosario.md](#), entrada Grounded generation (generación fundamentada)

Una sigla, dos sistemas: el RAG de Lewis (2020) y el del Taller 2

- La sigla la acuñan Lewis et al. (2020), en Facebook AI Research: *retrieval-augmented generation*
- La idea: antes de escribir, el generador consulta una memoria externa de texto
- Esa memoria es Wikipedia partida en 21 M pasajes de 100 palabras, en un índice denso
- Dos piezas: el recuperador DPR, dos BERT, y el generador BART, que recibe pregunta y pasaje juntos
- Lewis entrena el BERT de la pregunta y BART de extremo a extremo; el pasaje es una variable latente
- El Taller 2 conserva las piezas y no entrena ninguna: recuperar y pegar al prompt



Fuente: Lewis et al. (2020), resumen, p. 1 (la sigla; Facebook AI Research); §1, p. 2 (DPR y BART); §2, p. 3 (el documento como variable latente); §2.2–§2.3, p. 3 (DPR: dos BERT_BASE; BART-large); §2.4, p. 4 (se ajustan el codificador de la pregunta y BART; índice y codificador de documentos congelados); §3, p. 4 (Wikipedia de diciembre de 2018, pasajes de 100 palabras, 21 M documentos); Gao et al. (2023), §II.A, p. 3 (Naive RAG); proceso/contexto/glosario.md, distinción «RAG de Lewis (2020) vs. RAG del curso». La tabla de dos filas es [AGREGADO - no está en las fuentes]

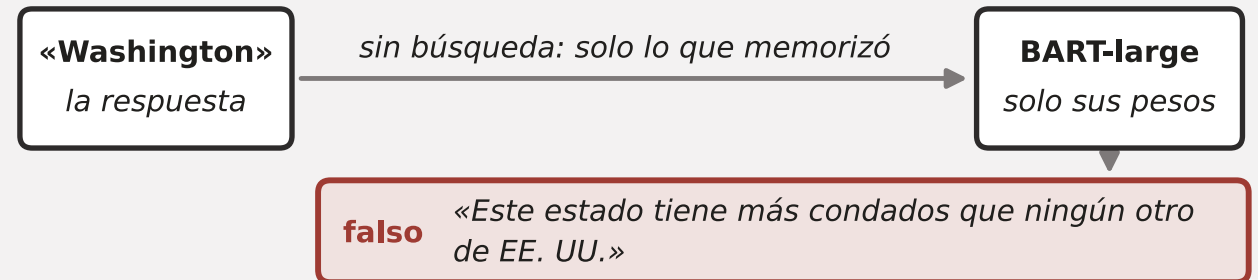
BART es el mismo generador, sin buscar

- BART-large: un transformer codificador-decodificador de 400 M parámetros
- Se preentrenó reconstruyendo texto al que se le había metido ruido
- Todo lo que sabe está en sus pesos: Lewis lo llama memoria paramétrica
- Solo, es la línea base de «libro cerrado»: responde con lo que memorizó
- RAG es ese mismo BART con los pasajes delante: memoria no paramétrica
- Mismo generador, misma tarea: la comparación aísla el efecto de buscar

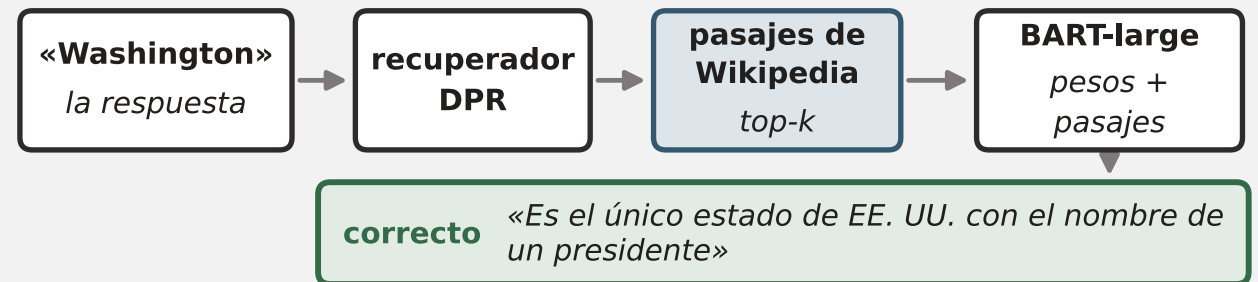
Misma pista, mismo BART: la única diferencia es buscar

Jeopardy al revés: dada la respuesta, escribir la pista

BART SOLO • libro cerrado



RAG • libro abierto

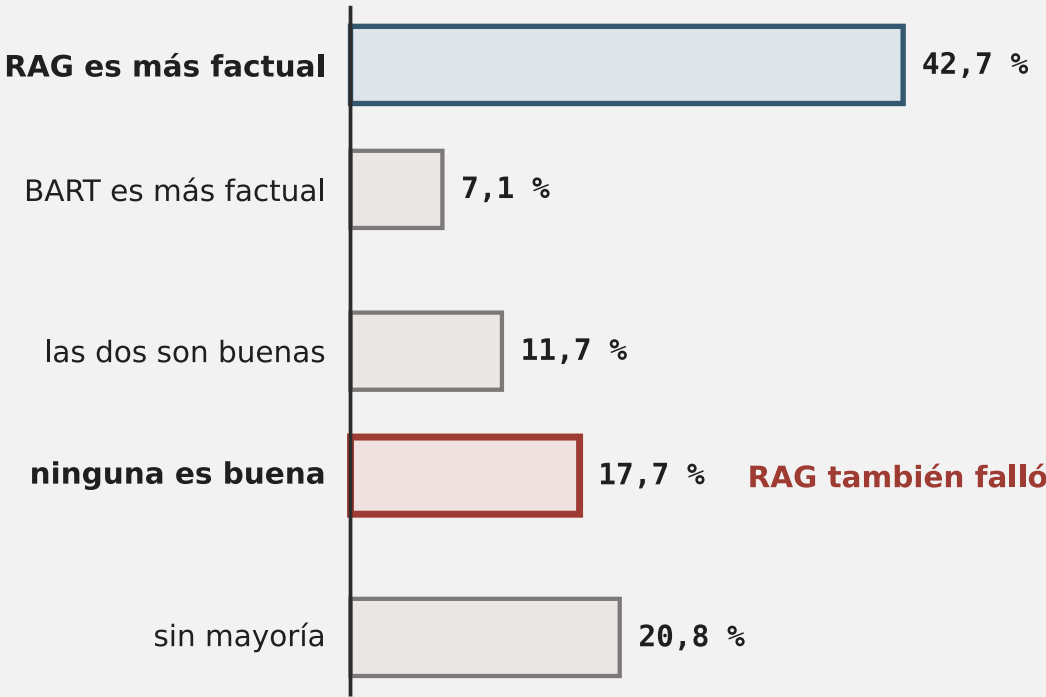


Salidas de la Tabla 3 de Lewis et al. (2020), traducidas

RAG alucina menos que BART. No deja de alucinar

- Personas compararon 452 pares de pistas, una de BART y otra de RAG: ¿cuál es más factual?
- RAG ganó en el 42,7 % y BART en el 7,1 %: leer el dato le gana a recordarlo
- Pero en el 17,7 % ninguna fue buena: con el pasaje delante, RAG también falló
- Primera puerta: el recuperador no trae el pasaje. En FEVER, 1 de cada 10 veces el artículo correcto no llega al top-10
- Segunda puerta: el generador también escribe desde sus pesos, no solo desde el pasaje
- RAG reduce las alucinaciones y las vuelve auditables; no las elimina

¿Cuál es más factual? 452 pares juzgados por personas



RAG también falló

Cada par cae en una sola fila: suman 100 %. Tarea Jeopardy, RAG-Token contra BART (Tabla 4, factualidad)

Fuente: Lewis et al. (2020), §3.3, p. 5 y §4.3, p. 6 (452 pares); Tabla 4, p. 8; §4.2, p. 6 («hallucinate less»); §4.4, p. 7 (FEVER, top-10 en el 90 %); §3.2, p. 5 y §4.3, p. 6 (conocimiento paramétrico); Alammar & Grootendorst (2024), cap. 8, pp. 225 y 250 («reduce»); proceso/contexto/rubrica_calidad.md. Las dos «puertas» son [AGREGADO - no está en las fuentes]

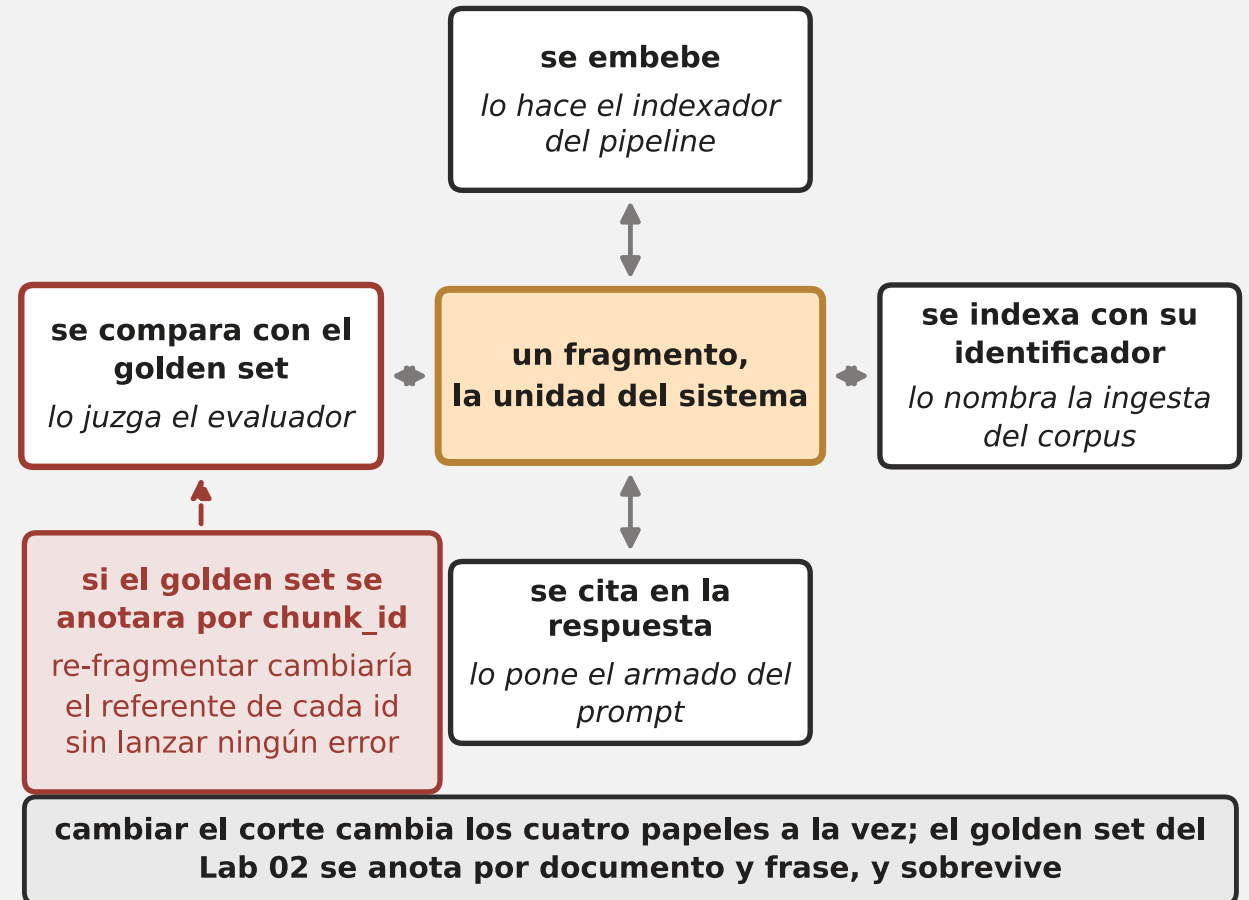
El fragmento es la unidad de todo

08

Un fragmento, cuatro papeles

- Se embebe: el indexador del pipeline convierte el texto de cada fragmento en vector
- Se indexa con su identificador: la ingesta lo nombra con su documento y su posición
- Se recupera y se cita: el armado del prompt pone el documento y el identificador de cada fragmento
- Se compara con el golden set: el evaluador busca en él el documento y la frase anotados
- Cambiar el corte cambia los cuatro papeles a la vez, y solo el primero se nota
- Por eso la fragmentación acota a la recuperación: ninguna búsqueda repara un mal corte

El fragmento atraviesa las cuatro etapas



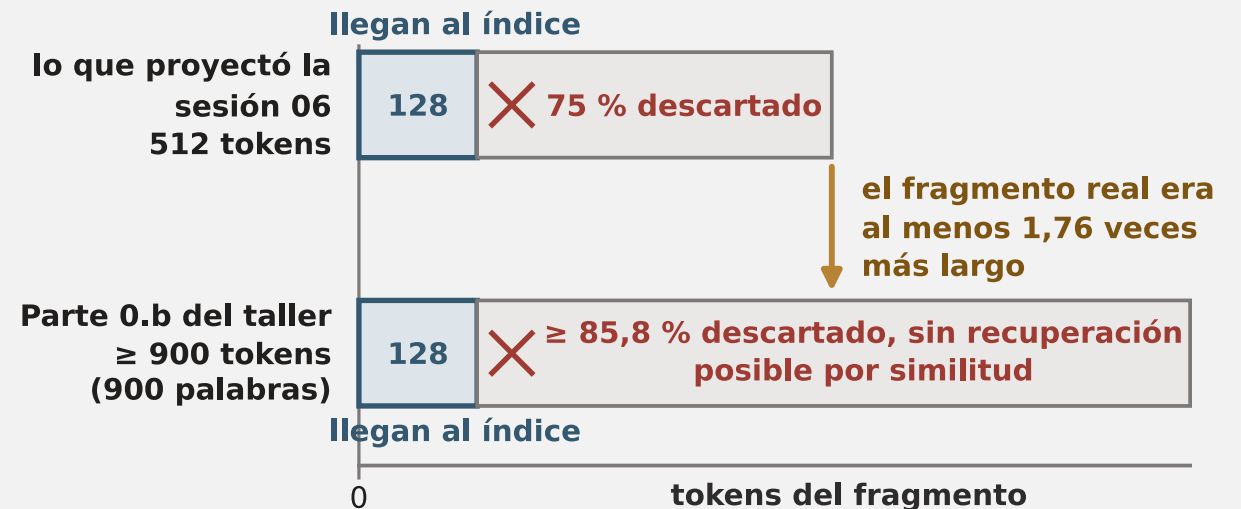
Fuente: [curso/labs/Lab-02-RAG-VectorSearch/](#): `rag_pipeline.py`, `RagPipeline.index` (el indexador) y `build_prompt` (el armado del prompt); `ingestion.py`, `load_corpus` (la ingesta, `id=f"{path.stem}-{idx:04d}"`); `golden_set_plantilla.json`; `evaluation.py`, `acierta()` (el evaluador) y su cabecera, punto 2 (por qué ya no se anota por `chunk_id`). Gao et al. (2023), §III.A.2, pp. 7–8 (granularidad). «El fragmento es la unidad de todo el sistema» es [AGREGADO - no está en las fuentes]; [proceso/contexto/glosario.md](#), entrada Chunking y distinción «Fragmentación vs. recuperación»

900 palabras contra un techo de 128 tokens

- El notebook de hoy vectoriza con el MiniLM multilingüe de la fila `embed_notebook_s2`: tope de 128 tokens
- Lo que pasa de 128, el modelo lo descarta sin avisar: ni excepción ni aviso
- Denle fragmentos de 900 palabras: toda palabra produce al menos un token, así que son al menos 900 tokens
- Llega al índice $\leq 14,2\%$ del fragmento y se descarta $\geq 85,8\%$. Nada falla
- La sesión 06 proyectó un 75 % descartado con la pareja 512/128. Con 900 palabras es peor
- Es la Parte 0.b del taller. El Lab 02 usa `bge-m3`, con tope de 8192, fragmenta en sus tokens y AVISA

El techo no cambia; lo que crece es lo que se descarta

`max_seq_tokens = 128`
`paraphrase-multilingual-MiniLM-L12-v2`
tabla semestral verificada 2026-08-27



**toda palabra produce al menos un token:
llega al índice $\leq 14,2\%$ del fragmento y nada falla;
el Lab 02 avisa; el modelo, no (Parte 0.b)**

Fuente: `curso/actividades/_fuentes/s2-mie.py`, Parte 2 (SentenceTransformer del MiniLM multilingüe); `fuentes/modelos/modelos-2026-1.json`, filas `embed_notebook_s2` (`max_seq_tokens = 128`) y `embed_local_multilingue` (8192), `verified_at 2026-08-27`; `curso/talleres/taller-02-rag-vector-search.md`, §«0.b – El fragmento que se corta a 128 (4 %)»; `curso/labs/Lab-02-RAG-VectorSearch/ingestion.py`, `load_corpus` (el aviso) y `rag_pipeline.py`, `FILA_EMBEDDINGS`; `slides_v2/sesion-06/contenido-base.md`, §10.4. La cota «una palabra $\geq$ un token» es [AGREGADO - no está en las fuentes]

El techo está en tokens. Si tu chunk_size no lo está, no sabes si te pasaste

- Notebook de hoy: 300 CARACTERES · regla del curso en el material: 500 TOKENS «aprox.»
- Notas teóricas: 512 TOKENS · Lab 02: 512 TOKENS de su propio modelo
- Techos: 128 TOKENS el del notebook, 8192 el de bge-m3. Cuatro artefactos, tres unidades
- Compararlos exige convertir, y convertir exige medir
- El Lab 02 lee el tope del modelo y fragmenta en su unidad: 512, sin pasar del tope – 2, con solape de un quinto
- La conducta: contar los tokens del fragmento más largo contra el tope antes de indexar. La Parte 1 pide justificarlo

Cuatro artefactos, tres unidades; los techos, en tokens			
artefacto	tamaño y solapamiento	unidad	
notebook de hoy	300 y 80	caracteres	
regla del curso (material)	500 y 100	tokens «aprox.»	
notas teóricas	512 y 64	tokens	
Lab 02, el del taller	512 y 102	tokens del modelo, leídos	
techo del notebook	128	tokens	los únicos límites duros
techo del Lab 02 (bge-m3)	8192	tokens	

300 caracteres contra 128 tokens: para saber si cabe, se cuenta el Lab 02 lee el tope, fragmenta en sus tokens y avisa si te pasas

Fuente: [curso/actividades/_fuentes/s2-mie.py](#), líneas 50 y 55; [curso/actividades/semana-2/s2-mie-explicacion.html](#), línea 77; [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §4.2, líneas 121–122, y §4.3, línea 150; [curso/labs/Lab-02-RAG-VectorSearch/ingestion.py](#), `load_corpus`; [curso/talleres/taller-02-rag-vector-search.md](#), §«Parte 1», punto 2; [fuentes/modelos/modelos-2026-1.json](#), filas `embed_notebook_s2` (128) y `embed_local_multilingue` (8192), `verified_at` 2026-08-27

Nada falla y todo responde. ¿Qué fracción del corpus puede encontrar tu sistema?

El escenario, el de la Parte 0.b del taller: 900 palabras por fragmento con el MiniLM del notebook de hoy, fila `embed_notebook_s2` (paraphrase-multilingual-MiniLM-L12-v2), ninguna excepción

A. Todo: el texto completo está en el payload y se recupera con la búsqueda

B. Como mucho el primer ~14 % de cada fragmento; el resto no es alcanzable por similitud

C. No se puede saber sin medir

Segunda pregunta, la que vale: ¿qué medirían para averiguarlo?

Y la tercera: ¿bajar el `chunk_size` o cambiar de modelo?

Fuente: `curso/actividades/_fuentes/s2-mie.py`, Parte 2 (el modelo del notebook); `curso/labs/Lab-02-RAG-VectorSearch/rag_pipeline.py`, `FILA_EMBEDDINGS` (la ruta local de la Parte 0 y la de `bge-m3`); `curso/talleres/taller-02-rag-vector-search.md`, §«Parte 0.b» (el escenario es el que esa parte hace reproducir); `fuentes/modelos/modelos-2026-1.json`, filas `embed_notebook_s2` (`max_seq_tokens` = 128) y `embed_local_multilingue` (8192), ambas `verified_at` 2026-08-27; `proceso/contexto/glosario.md`, §Términos, líneas 167–168, y distinción vigilada «techo del modelo vs. tamaño del fragmento vs. ventana de contexto». Las entradas se leen en voz alta después de recoger las respuestas, no antes

Cinco estrategias, un solo compromiso

- Fijo: simple y predecible; corta oraciones a la mitad
- Sliding window: fijo con más solapamiento; infla el índice (siguiente diapositiva)
- Semántico o estructural: respeta párrafos y secciones; exige documentos bien estructurados
- Jerárquico: fragmentos pequeños para buscar que apuntan a su sección padre para el contexto
- Contextual: contexto añadido al fragmento — y su versión gratis y con fuente: el título del documento
- La regla que sí se puede defender no es un número: un párrafo es un fragmento; si son largos, de 3 a 8 oraciones

Cómo corta cada estrategia y qué cuesta

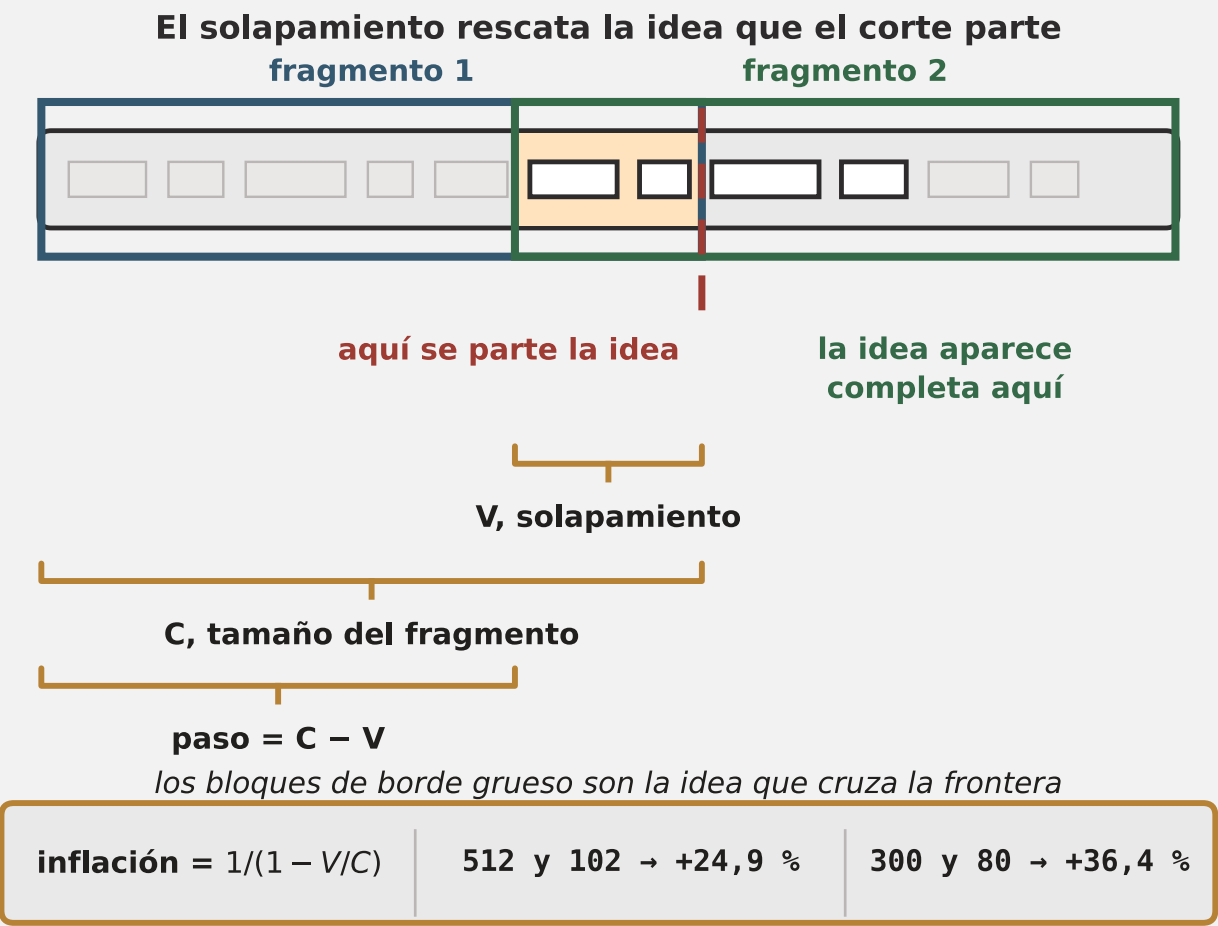
estrategia	cómo corta	qué cuesta
fijo	cada N unidades	corta ideas a la mitad
sliding window	fijo con solapamiento	índice más grande
semántico o estructural	por párrafos y secciones	exige documentos bien estructurados
jerárquico	pequeño para buscar, padre para el contexto	más piezas que mantener
contextual	contexto añadido al fragmento (título, o generado por un LLM)	cero llamadas si es el título, una por fragmento si lo genera un LLM <i>y alarga el fragmento, así que empeora el truncado</i>

ninguna de las cinco cierra el compromiso entre completitud semántica y longitud

Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §4.1, líneas 112–114, y §4.2, líneas 118–142; [curso/actividades/semana-2/s2-mie-explicacion.html](#), líneas 66–75; Gao et al. (2023), §III.A.2, p. 7 (el compromiso de granularidad), §III.B.1, p. 8 (tamaño fijo, ventana deslizante, Small2Big y la frase del compromiso) y §III.B.3, p. 8 (índice jerárquico); Alammar & Grootendorst (2024), cap. 8, §«Chunking long texts», pp. 236–237 (la regla estructural y el título del documento como contexto)

El solapamiento: qué salva y cuánto cuesta, exactamente

- Sin solapamiento, una idea que cruza la frontera queda mutilada en los dos fragmentos
- Con solapamiento aparece completa en al menos uno. Es un parche a un corte arbitrario
- El costo es exacto: la inflación del índice es $C/(C - V)$, es decir $1/(1 - V/C)$
- Lab 02 por defecto, 512/102 en tokens ($V = C/5$ con división entera): +24,9 % · notas, 512/64: +14,3 %
- Regla del curso, 500/100: +25,0 % · el notebook de hoy, 300/80, se sale de la horquilla del material: +36,4 %
- Regla del curso: el baseline es fijo con solapamiento, y lo sofisticado debe ganarle midiendo



Fuente: [curso/actividades/semana-2/s2-mie-explicacion.html](#), línea 121 (autocomprobación: índice ~15-25 % más grande) y líneas 77-78 (regla del curso); [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §4.2, líneas 121-122; [curso/labs/Lab-02-RAG-VectorSearch/ingestion.py](#), load_corpus y CHUNK_TOKENS_POR_DEFECTO (chunk_tokens = 512 sin pasar del tope - 2, overlap_tokens = chunk_tokens // 5); [curso/actividades/_fuentes/s2-mie.py](#), línea 55, notas docentes 13-14 y «Mira los cortes», líneas 79-81; Gao et al. (2023), §III.B.1, p. 8; Alammari & Grootendorst (2024), cap. 8, §«Chunking long texts», pp. 236-238 (Figuras 8-8 y 8-10, no incrustadas). La fórmula $C/(C - V)$ y el +24,9 % del Lab 02 son [AGREGADO - no está en las fuentes]

Golden set anotado por chunk_id. Cambias de chunking, vuelves a medir y bajan los números. ¿Qué aprendiste?

Es la Opción A del Taller 2 con un golden set que anota la respuesta correcta como una lista de chunk_id

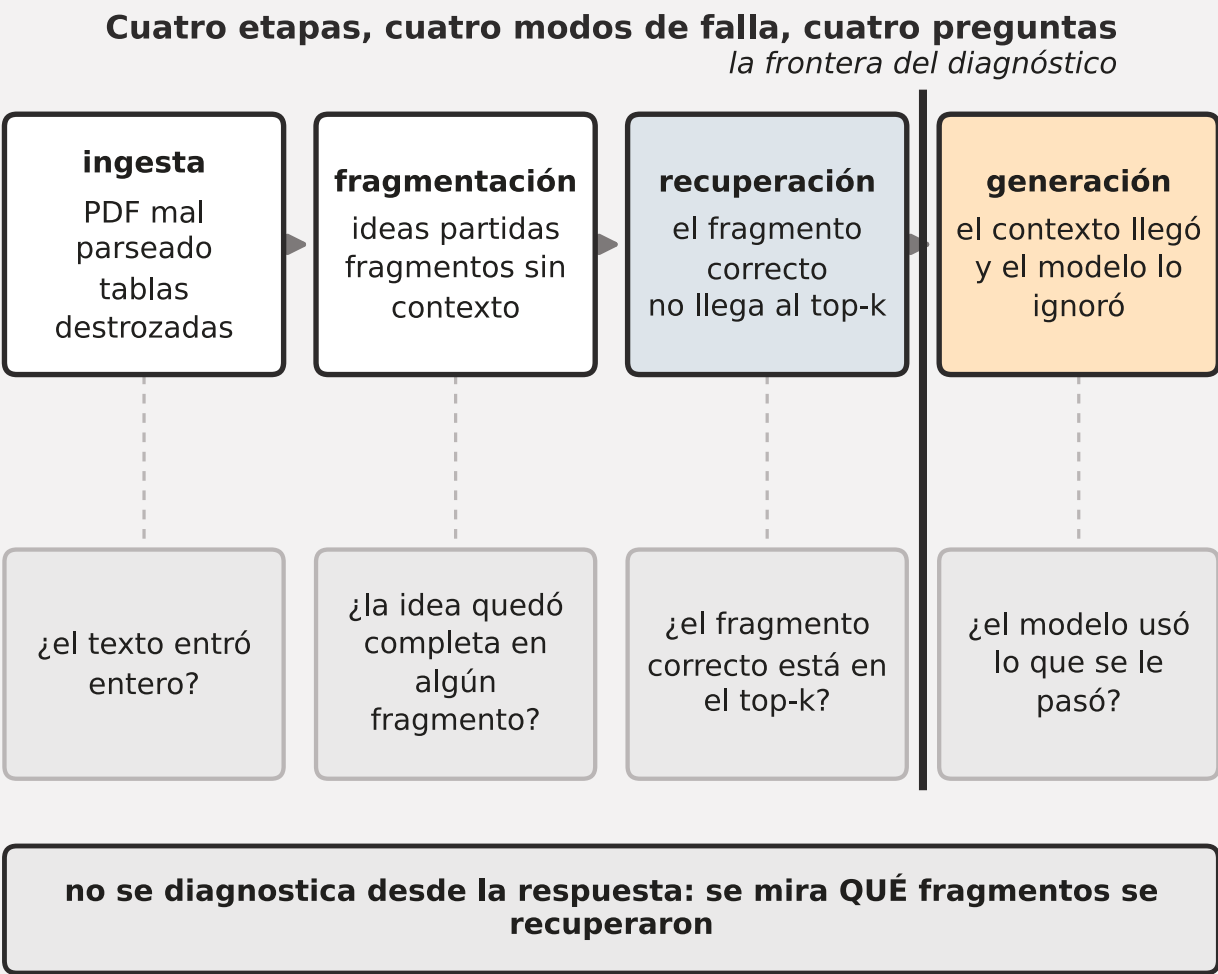
- A. Que la nueva estrategia es peor: para eso se compara contra el baseline
- B. Posiblemente nada: esos chunk_id los fabricó el chunker anterior
- C. Que hay que subir el top-k para compensar

Pista: ¿quién inventó los identificadores que dicen cuál es la respuesta correcta?

Segunda pregunta, la que se cobra el sábado: ¿cómo habría que anotar para que la comparación valga?

El chunk culpable — 15 minutos en grupos

- Tres fallas reales: respuesta incompleta, respuesta de la sección equivocada, «no encuentro» cuando sí existe
- Para cada una: ¿falló la recuperación o falló la generación?
- ¿Qué cambiarían primero: el chunking, el top-k o el prompt?
- Ingesta: PDFs mal parseados, tablas destrozadas · Fragmentación: ideas partidas, «él afirmó que...»
- Recuperación: el fragmento correcto existe y no llega al top-k · Generación: llegó y el modelo lo ignoró
- Este diagnóstico es la Parte 2.c del Taller 2 —10 %, con evidencia—, y mañana le ponemos métricas



Fuente: [curso/actividades/semana-2/s2-mie-explicacion.html](#), líneas 80–91 (la actividad, sus tres fallas y las dos preguntas, literales) y líneas 105–113 (las cuatro cajas de fallo); [curso/talleres/taller-02-rag-vector-search.md](#), §«Parte 2.c – Análisis de fallos» y §«Criterios de evaluación» (10 %); Gao et al. (2023), §II.A, p. 3; [curso/labs/Lab-02-RAG-VectorSearch/ingestion.py](#), `read_pdf` (inserta `[page=N]` en el texto) y `parece_escaneado` (el PDF sin texto útil no se indexa, y avisa). Las tres fallas concretas las produce el profesor con el corpus del notebook: [AGREGADO - no está en las fuentes]

Cómo se busca: léxica, densa e híbrida

08

Lo que la densa no recupera

- Si la respuesta no está en el corpus, igual devuelve k fragmentos con distancias de aspecto sano
- No existe el «no encontré»: el umbral máximo de distancia lo pone quien diseña, no el modelo
- Frase exacta, código de artículo, identificador: eso es coincidencia literal, y la densa no la hace
- Por eso el libro del sílabo RECOMIENDA híbrida, no densa sola. No es una opción de lujo
- Y el límite que no se ve: fuera de su dominio de entrenamiento, rinde peor
- El idioma es un dominio: por eso el Lab 02 usa bge-m3, un modelo multilingüe

Fuente: Alammar & Grootendorst (2024), cap. 8, §«Caveats of dense retrieval», pp. 234–235; fuentes/modelos/modelos-2026-1.json, filas `embed_local_multilingue` (bge-m3, `multilingual = true`, la del Lab 02) y `embed_local_rapido` (`multilingual = false`), ambas `verified_at` 2026-08-27; curso/labs/Lab-02-RAG-VectorSearch/rag_pipeline.py, FILA_EMBEDDINGS; curso/talleres/taller-02-rag-vector-search.md, §«Con qué modelo corre cada parte» (la fila se declara por su id con su `verified_at`) y §«Parte 2.a» (al menos 2 negativas); proceso/contexto/glosario.md, entradas `Distance threshold` (umbral máximo de distancia) y `Domain mismatch` (desajuste de dominio). «No es una opción de lujo» es [AGREGADO - no está en las fuentes]

BM25 busca las palabras que escribiste

- BM25 puntúa cada fragmento contra la consulta sin ningún modelo: parte la consulta en términos y suma una contribución por término
- IDF: cuán RARO es el término en el corpus. «de» está en todos los fragmentos y casi no pesa; una palabra que está en pocos, mucho
- Esa rareza se multiplica por la frecuencia del término en el fragmento, que SATURA: diez apariciones no valen diez veces una
- Y se descuenta el LARGO: $|frag|/avgdl$ compara el fragmento con la longitud media del corpus
- Un término que no aparece suma 0; si no coincide ninguno, el puntaje es 0: BM25 sí sabe decir «nada»
- k_1 y b son parámetros de BM25, no el top-k. Re-fragmentar cambia avgdl y los IDF: cambian TODOS los puntajes

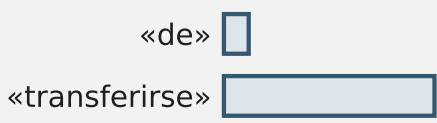
Cómo puntúa BM25 un fragmento

$$BM25(q, frag) = \sum_{t \in q} IDF(t) \cdot S(t, frag)$$
$$S(t, frag) = \frac{tf(k_1 + 1)}{tf + k_1 (1 - b + b |frag|/avgdl)}$$

un término que no está en el fragmento suma 0

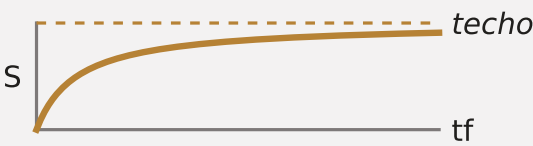
1 • IDF: cuán rara es la palabra

«de» casi no pesa; «transferirse», mucho



2 • la frecuencia satura

diez apariciones no valen diez veces una



3 • se descuenta el largo

más largo que la media, más azar a favor



Fuente: fuentes/notas-teoricas/s2-rag-y-vector-search.md, §5.1, líneas 160 y 163 (la fórmula, reescrita con frag y tf); curso/actividades/_fuentes/s2-mie.py, DOCS, politica_vacaciones.md («transferirse», del corpus del notebook); Alammar & Grootendorst (2024), cap. 8, p. 234 (ejemplo trabajado de BM25; «toy example» del propio libro); proceso/contexto/glosario.md, entradas BM25 y k_1 , b . La lectura de la fórmula en tres piezas, el «suma 0» y la consecuencia de avgdl son [AGREGADO - no está en las fuentes]

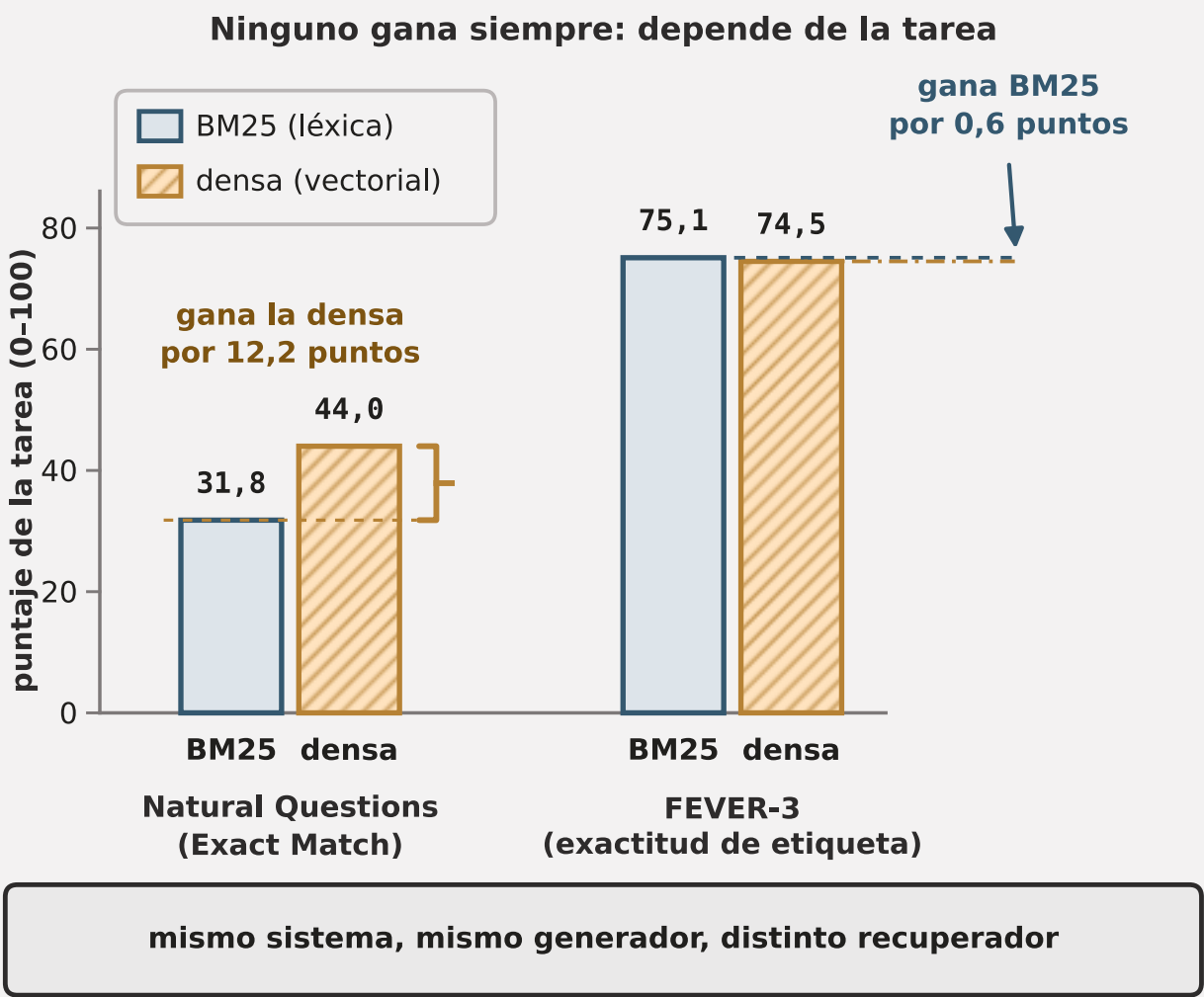
BM25 o densa: qué quiere decir que un fragmento salga primero

- La búsqueda densa es la de toda la semana: consulta y fragmentos pasan por el modelo de embeddings —bge-m3 en el Lab 02— y se ordenan por coseno
- BM25 no usa modelo: es una evolución de TF-IDF que cuenta coincidencias de términos, sin GPU y sin entrenar nada
- Primero en BM25 quiere decir «comparte contigo las palabras más raras»; primero en la densa, «dice algo parecido»
- BM25 acierta con siglas, códigos y nombres propios, y no ve sinónimos: «perro» y «can» son términos distintos
- La densa entiende sinónimos y paráfrasis, y puede fallar con términos técnicos exactos, identificadores y números
- Sus puntajes no se comparan: BM25 no tiene cota y da 0 si nada coincide; el coseno vive en $[-1, 1]$ y siempre devuelve k

Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §5.1, líneas 160 y 166–168, y §5.2, líneas 172–176; [curso/labs/Lab-02-RAG-VectorSearch/rag_pipeline.py](#), CodificadorH200 (bge-m3, vectores normalizados); [fuentes/modelos/modelos-2026-1.json](#), fila `embed_local_multilingue` (verified_at 2026-08-27); Alammari & Grootendorst (2024), cap. 8, p. 234 (ejemplo trabajado de BM25); Gao et al. (2023), §III.D, p. 9; [proceso/contexto/glosario.md](#), entradas Dense retrieval y BM25. Las dos lecturas de «salir primero» son [AGREGADO - no está en las fuentes]

Cuál recuperador gana depende de la tarea

- Lewis sustituyó su recuperador denso por BM25, con todo lo demás igual, y midió
- Natural Questions (Exact Match): 31,8 con BM25 frente a 44,0 denso — 12,2 puntos
- TriviaQA 44,1 vs. 55,8 · WebQuestions 36,6 vs. 44,9 · CuratedTrec 33,8 vs. 53,4
- FEVER-3: 75,1 con BM25 frente a 74,5 denso. FEVER-2: 91,6 frente a 90,6
- Los autores lo explican: las afirmaciones de FEVER son «heavily entity-centric»
- Lo que la tabla NO demuestra: que combinarlos sea mejor. Lewis no midió híbrido



Fuente: Lewis et al. (2020), §4.5, p. 7 (se sustituye el recuperador denso por BM25; «FEVER claims are heavily entity-centric») y Tabla 6, p. 8 (las seis cifras; la tabla se redibuja, no se incrusta); proceso/contexto/glosario.md, distinción vigilada «BM25 vs. denso: no hay una mejor, hay una mejor para la tarea»

RRF combina votos, no puntajes

- RRF es *Reciprocal Rank Fusion*, fusión por rango recíproco: junta la lista de BM25 y la densa en una sola
- Los puntajes no se pueden sumar: el coseno vive en $[-1, 1]$ y BM25 no tiene cota. RRF usa POSICIONES
- $\text{rank}_{\text{bm25}}$ es el puesto del documento en la lista de BM25; $\text{rank}_{\text{denso}}$, su puesto en la de la búsqueda densa. El primero es 1
- Cada lista le da $1/(k_{\text{RRF}} + \text{rank})$ —el recíproco del rango— y se suman; si una lista no lo trajo, no suma
- 2.º en las dos listas: $0,0323 \cdot 1.º$ en una y ausente en la otra: 0,0164. Premia el ACUERDO
- $k_{\text{RRF}} = 60$ por convención: aplana la curva, del puesto 1 al 10 hay un 13 %; con 0 sería un factor 10

Segundo en las dos listas le gana a primero en una sola

documento	puesto en BM25	puesto en denso	puntaje RRF	
A	2	2	0,0323	segundo en las dos listas casi el doble de puntaje primero en una sola
B	3	3	0,0317	
C	10	1	0,0307	
D	1	ausente	0,0164	

$$\text{rrf} = \frac{1}{k_{\text{RRF}} + \text{rank}_{\text{bm25}}} + \frac{1}{k_{\text{RRF}} + \text{rank}_{\text{denso}}}$$

rank: el puesto del documento en esa lista (1 = el primero)
el 60 es una convención de la práctica, no un resultado medido

Fuente: fuentes/notas-teoricas/s2-rag-y-vector-search.md, §5.3, líneas 184 y 187 (la fórmula y el 60 como valor estándar; las notas escriben k donde el glosario pide k_{RRF}); Gao et al. (2023), §III.D.1, p. 9 (Mix/hybrid Retrieval: rasgos de relevancia complementarios y entidades raras); slides_v2/sesion-06/contenido-base.md, §8.2 (el puntaje denso vive en $[-1, 1]$). Los cuatro puntajes son la fórmula citada evaluada en cuatro parejas de rangos: [AGREGADO - no está en las fuentes]; Gao et al. (2023), §III.D.1, p. 9, el porqué de combinar, va en las notas

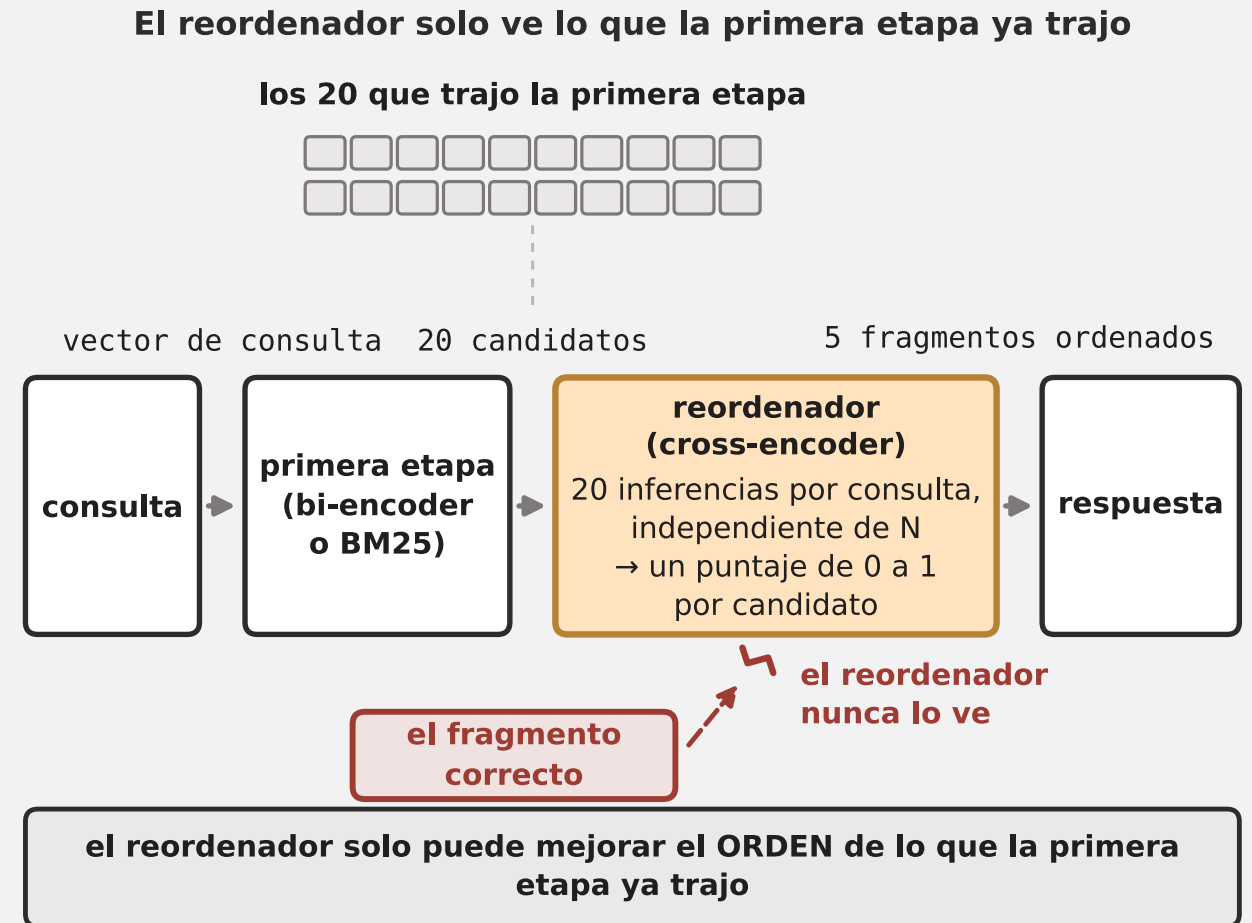
Para qué reordenar: la primera etapa es rápida porque no lee la pregunta

- La primera etapa —densa, BM25 o híbrida— compara la consulta con TODO el corpus, así que tiene que ser barata
- La densa es barata porque el vector de cada fragmento se calcula al indexar, antes de que exista la pregunta
- El precio: el orden del top-k es aproximado, y el LLM solo lee los pocos fragmentos que le pasamos, en ese orden
- Un cross-encoder lee la consulta y el fragmento JUNTOS: cada palabra de la pregunta se cruza con cada palabra del fragmento
- Por eso es más preciso —unos 7 puntos de correlación, sesión 06— y no se puede precalcular: una inferencia por par
- Sobre todo el corpus sería impagable; sobre los 20 que ya trajo la primera etapa, cuesta 20 inferencias

Fuente: [slides_v2/sesion-06/sesion-06.qmd](#), diapositiva «Bi-encoder y cross-encoder: 65 horas o 5 segundos»; [slides_v2/sesion-06/contenido-base.md](#), §7, sobre Reimers & Gurevych (2019), §1, pp. 1–2, y §4.3, p. 5, Tabla 3; [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §5.4, líneas 193–201; Gao et al. (2023), §IV.A, p. 10 («lost in the middle»); [proceso/contexto/glosario.md](#), entradas Bi-encoder, Reranking y Two-stage retrieval. «Cada palabra de la pregunta se cruza con cada palabra del fragmento» es [AGREGADO - no está en las fuentes]

El reranking reordena; no recupera

- Cómo se hace, en tres pasos: la primera etapa trae top-20; el cross-encoder puntúa cada par (consulta, candidato); se reordena y se pasan los 5 mejores al prompt
- El puntaje va de 0 a 1 y es clasificación, no similitud: importa el ORDEN, no la magnitud
- Cuesta 20 inferencias por consulta, vengan de 10 000 fragmentos o de un millón: no crece con el corpus
- Su techo es la lista que recibe: si el fragmento correcto no está entre los 20, nunca lo mira
- Si reordenar no mejora nada, miren la lista de 20 ANTES de reordenar: el problema puede estar antes
- En el paper obligatorio no hizo falta; en el Taller 2 es la Opción C, y su ganancia se mide



Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §5.4, líneas 193–201 (dos etapas top-20 → top-5); Gao et al. (2023), §IV.A.1, p. 10 (el reordenamiento como realce y filtro); Lewis et al. (2020), §4.1, p. 5 («neither a re-ranker nor extractive reader is necessary»); [slides_v2/sesion-06/contenido-base.md](#), §7, sobre Reimers & Gurevych (2019), §1, pp. 1–2, y §4.3, p. 5, Tabla 3; Alammar & Grootendorst (2024), cap. 8, §«How reranking models work», p. 244 (puntaje de relevancia 0–1; monoBERT); [fuentes/modelos/modelos-2026-1.json](#), clave rerankers, filas rerank_local_ingles y rerank_local_multilingue (verified_at 2026-09-19; max_seq_tokens sin declarar en las dos model cards; la multilingüe sin cifras publicadas). La aritmética N frente a 20 por consulta es [AGREGADO - no está en las fuentes]

AI notebook: 55 minutos

08

Más contexto no es mejor contexto

- El top-k es perilla de consulta: se cambia sin reindexar y sin reentrenar. El notebook usa 3
- En el paper obligatorio, subirlo mejora una variante y la otra tiene un máximo en 10
- Gao lo llama error frecuente: creer que cuantos más fragmentos se concatenen, mejor
- «Lost in the middle»: el modelo atiende al principio y al final y olvida el medio
- Por eso el ORDEN en que se pegan los fragmentos importa, y reordenar no es cosmético
- Subir el top-k es gratis para el índice y caro para el modelo

El prompt del baseline hace tres cosas

- Delimitar el contexto con etiquetas separa el DATO de la INSTRUCCIÓN
- «Responde SOLO con base en el contexto» fuerza la atribución y hace auditable la respuesta
- Autorizar el «no sé» con UNA frase fija —ABSTENCION, la misma del Lab 02 y del golden set— convierte la abstención en salida válida y detectable, no en fallo

```
PLANTILLA = """Eres un asistente de politicas internas de NimbusSoft.  
Responde SOLO con base en el contexto siguiente. Si la  
respuesta no esta en el contexto, di exactamente:  
"El corpus no contiene informacion suficiente."  
Cita el fragmento [n] que uses.
```

```
<contexto>  
{contexto}  
</contexto>
```

```
Pregunta: {pregunta}  
Respuesta: """
```

Fuente: curso/actividades/_fuentes/s2-mie.py, línea 133 (ABSTENCION) y líneas 135–145 (PLANTILLA: fuente canónica de los dos notebooks; saltos de línea reajustados a 72 columnas y acentos retirados, sin cambiar ninguna palabra ni identificador), se_abstuvo (líneas 169–171) y líneas 218–221; curso/labs/Lab-02-RAG-VectorSearch/rag_pipeline.py, ABSTENCION, se_abstuvo y build_prompt (la misma frase; la detección es por contención normalizada, no por igualdad de cadena); curso/actividades/semana-2/s2-mie-explicacion.html, líneas 126–127; Gao et al. (2023), §VI.C.2, p. 12 (Negative Rejection); Alammar & Grootendorst (2024), cap. 8, §«The RAG prompt», pp. 253–254 (la plantilla del libro no autoriza la abstención, y su salida publicada especula). Sin credenciales: la clave se lee del entorno (verificado 2026-09-02 y releído el 2026-09-20 sobre la versión del 19)

El sábado: fragmentar con parámetros que sepas defender

- Taller 2 — sistema RAG sobre base vectorial, 25 %, sábado 26 de septiembre
- Parte 1: fragmentar con parámetros explícitos y justificados. Hoy vieron CONTRA QUÉ se justifican
- Parte 3, cuatro opciones: tres salen de hoy — otra fragmentación, BM25 con RRF, reordenamiento
- La cuarta es de evaluación y es de mañana. RRF se practica mañana; BM25 y el reordenador, en ninguna sesión
- Opción C: la tabla semestral tiene dos filas de reordenadores; la multilingüe no publica cifras — se mide, no se supone
- Mañana: ¿este RAG es bueno? Hit Rate, MRR, Recall@k y golden sets. Hoy no medimos nada